

Vivado 使用入门

简介:

一个典型的，用 Vivado 进行数字逻辑电路设计的过程包括：创建模块，创建用户约束文件，创建 Vivado 工程，插入创建的模块，声明创建的约束文件，随意地运行动作的仿真，对创建的模块进行综合，执行设计，产生位文件，最后将文件下载到硬件上验证设计的正确性等步骤。

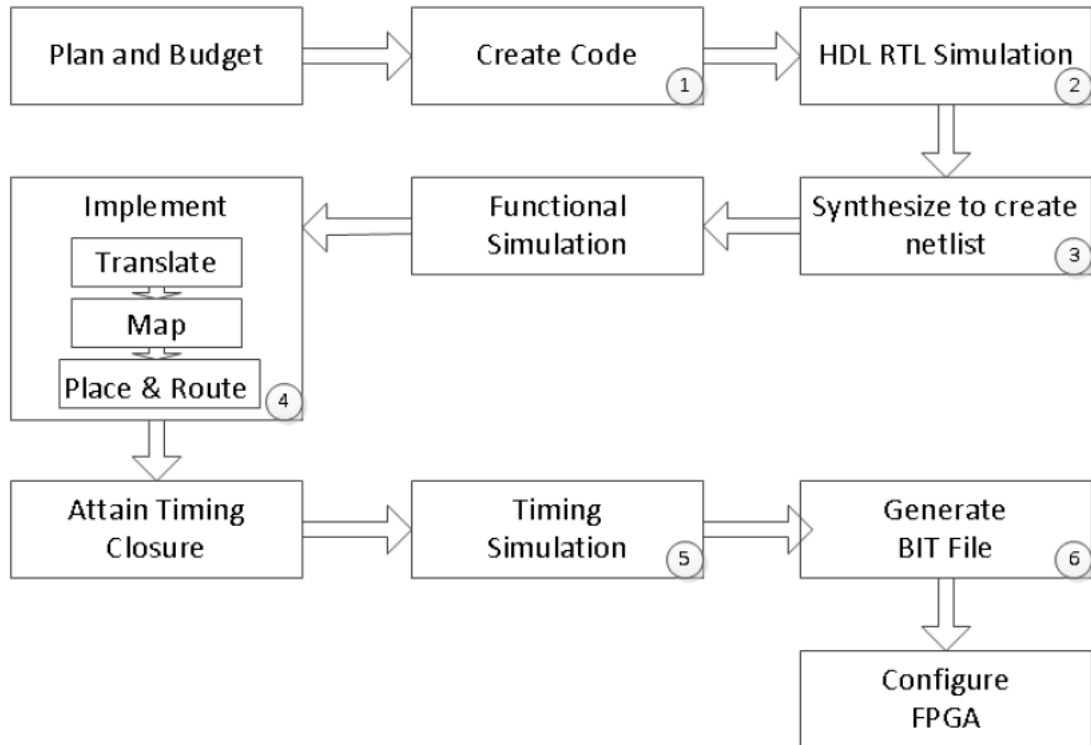


Figure 1. A typical design flow

完成一个实际电路的实例

一、新建工程

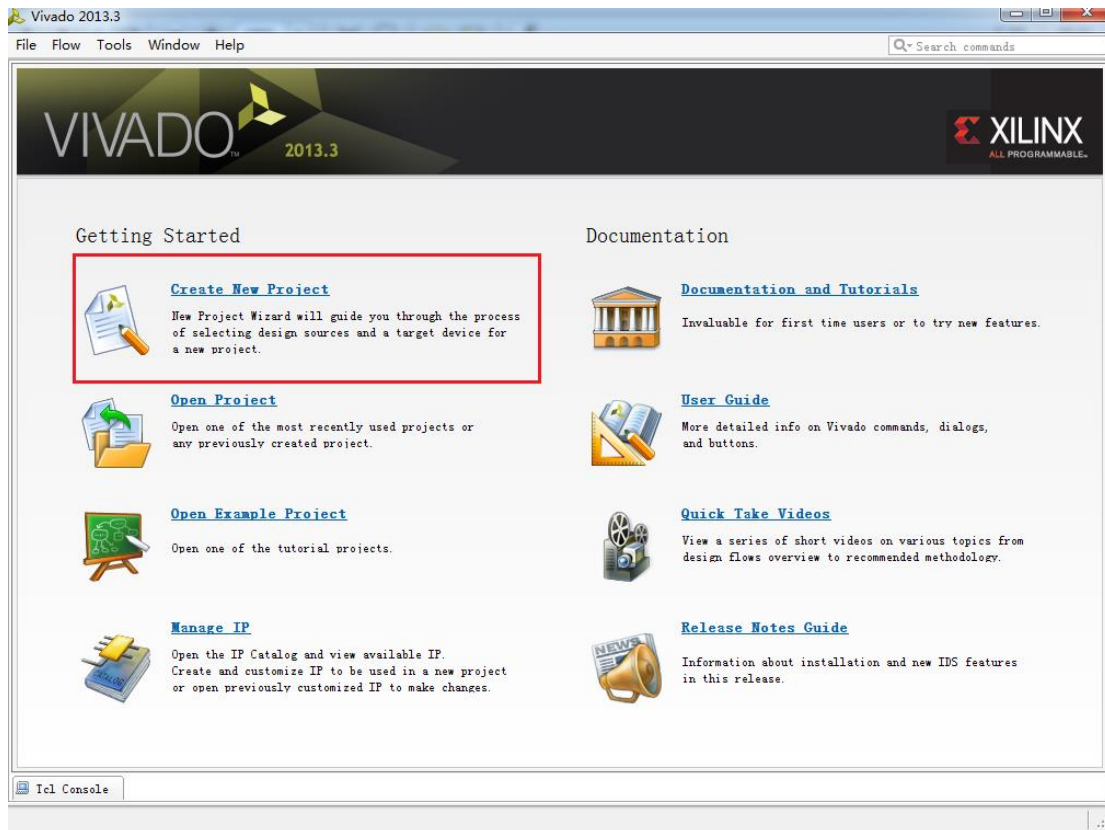
步骤如下:

1 打开 Vivado

Start > All Programs > Xilinx Design Tools > Vivado 2013.3 > Vivado 2013.3 或双



击桌面图标，显示如下界面：



“Creating New Project”：建立新工程导航，用于建立各种类型的工程。

“Open Project”：打开一个已有的工程。

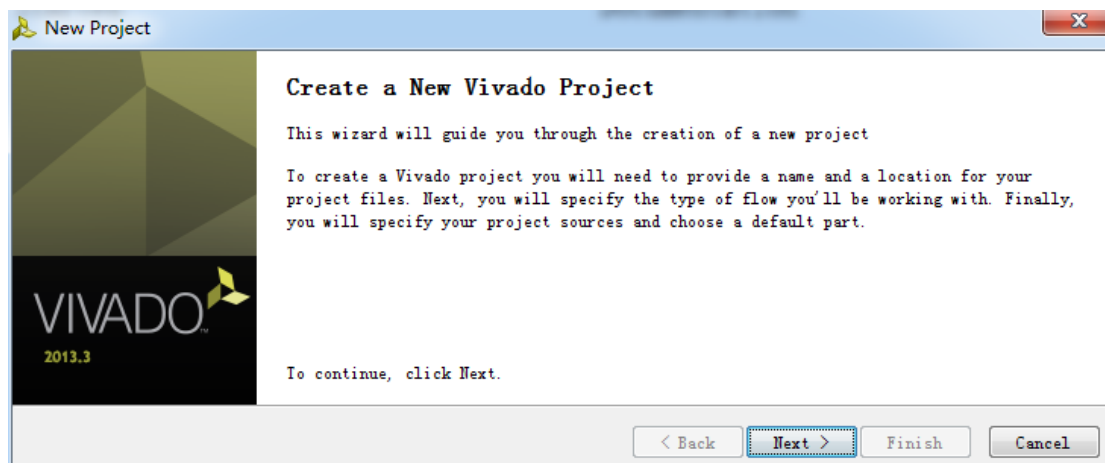
“Open Example Project”：打开示例工程。

“Documentation and Tutorials”：文件夹和说明书。可以打开Xilinx使用说明书和部分设计数据。

“User Guide”：打开Vivado用户指南。

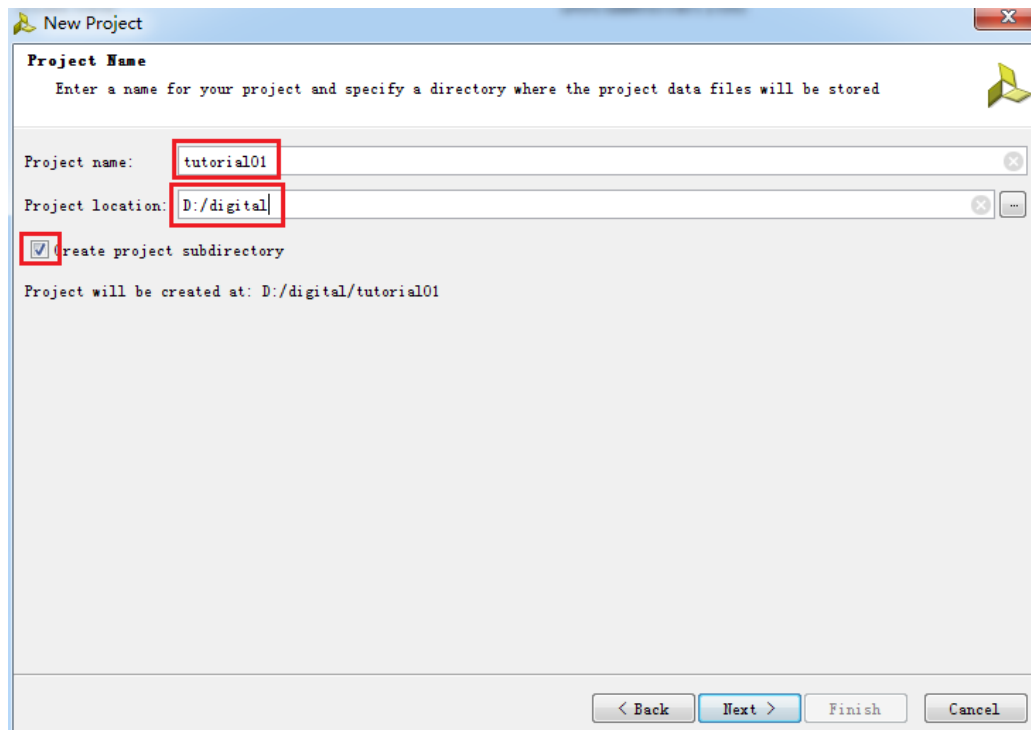
“Quick Take Videos”：打开Xilinx视频说明书。

2 点击 “**Create New Project**”，开始新建工程向导。弹出下图

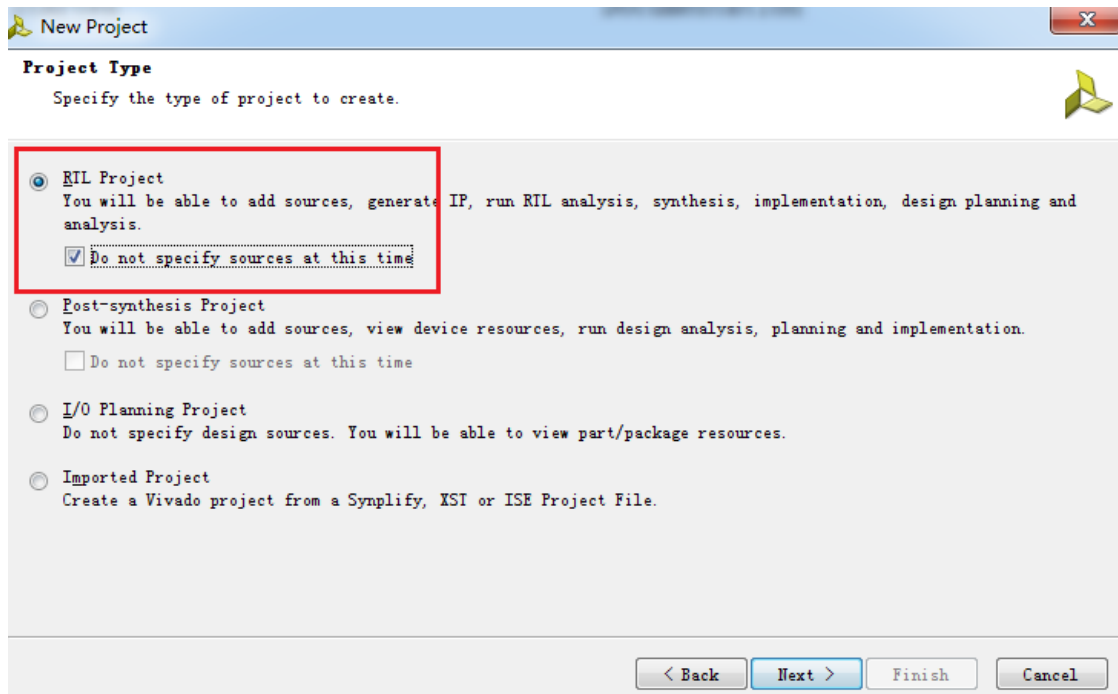


点击“Next”，进入设置此次所建项目的名称对话框。第一次实验前，请为本课程所有的项目新建一个文件夹，如“digital”，以后所有本课程相关项目均放在此文件夹下，在“project location”栏选择此文件夹。接着，为本项目取名，如“tutorial01”，勾选“Create Project Subdirectory”，在当前目录下为本工程新建一个同工程名的子目录，保存工程所有数据文件。

注意：工程名称和存储路径中不能出现中文和空格，建议工程名称和路径名称都是以字母开头，由字母、数字、下划线来组成。

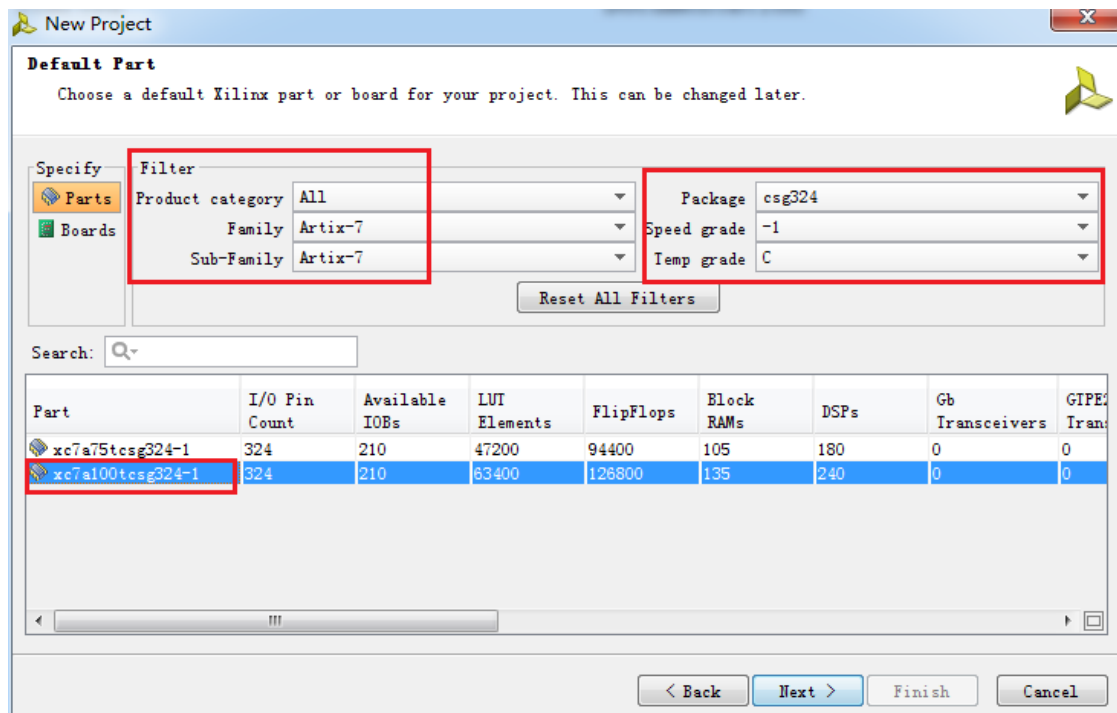


点击“Next”。进入工程类型界面，指明工程类型，这一步定义了工程源文件的类型。选择“RTL Project”。勾选“Do not specify sources at this time”，勾选该选项是为了跳过在新建工程的过程中添加设计源文件的过程。

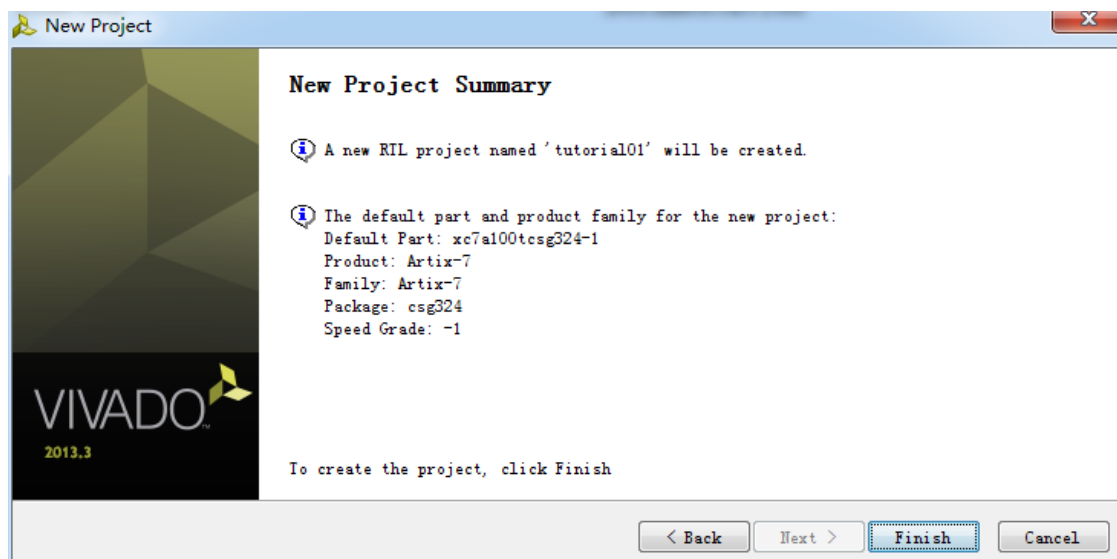


点击“Next”。进行目标器件的选择，根据实验平台选择相应的FPGA器件。本实验使用的是Xilinx公司的Nexys4开发板，此开发板上的FPGA为Artix-7系列芯片，即“Family”和“Subfamily”均为Artix-7，封装形式“Package”为csg324，速度等级“Speed grade”为-1，温度等级“Temp Grade”为C。在出现的两个器件中，选择xc7a100tcsg324-1的器件。

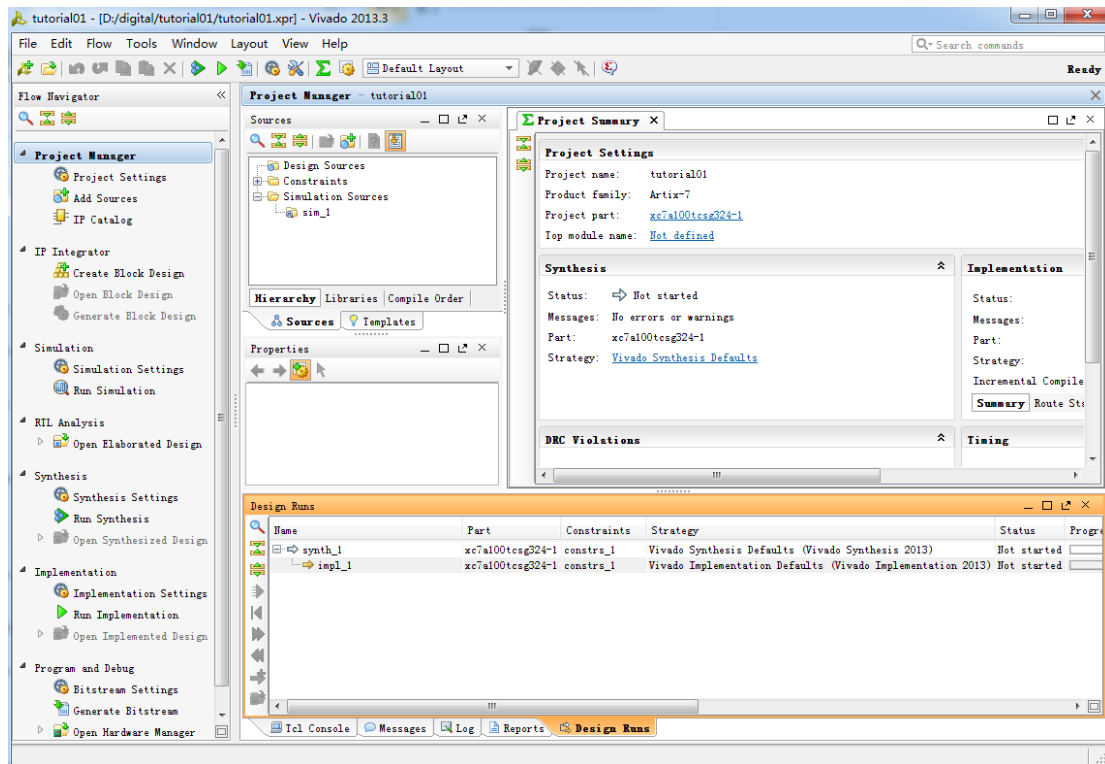
注：实验室也提供zedboard的开发板，如果选用的是zedboard的开发板，此开发板上的FPGA为Zynq-7000系列芯片，即“Family”和“Subfamily”均为Zynq-7000，封装形式“Package”为clg484，速度等级“Speed grade”为-1。



点击“Next”。进入新建工程总结界面，确认相关信息与设计所用的FPGA器件信息是否一致，一致请点击“Finish”，不一致，请返回上一步修改。

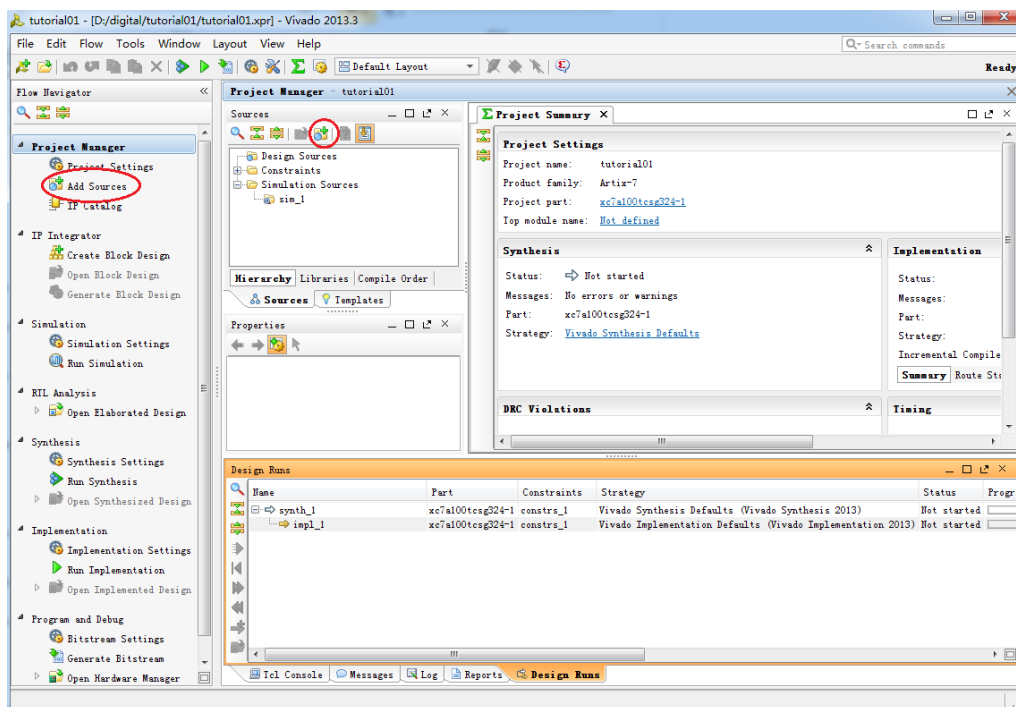


完成后，进入空白的Vivado工程界面，如图，完成新建工程过程。

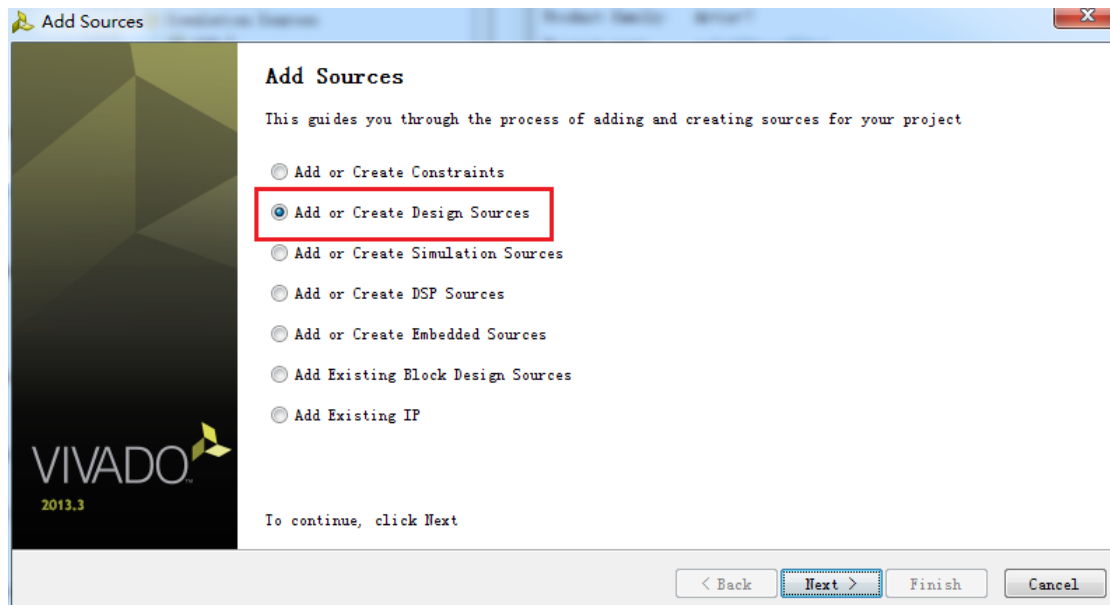


二、输入设计文件

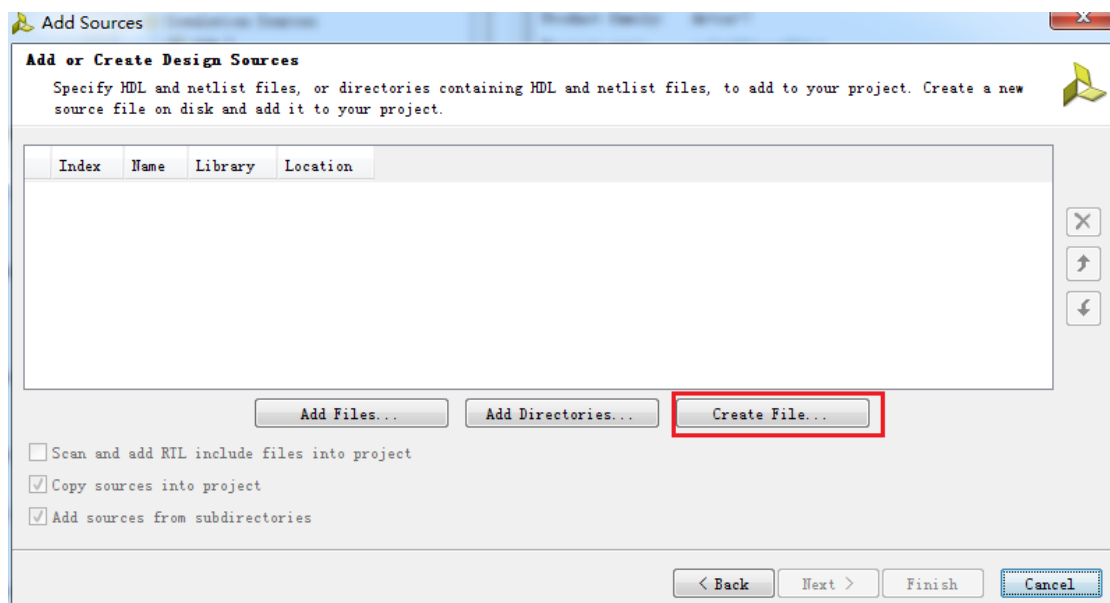
- 1、如下图所示，点击Flow Navigator下的Project Manager->Add Sources或中间Sources中的Add Sources对话框，打开添加设计文件对话框。



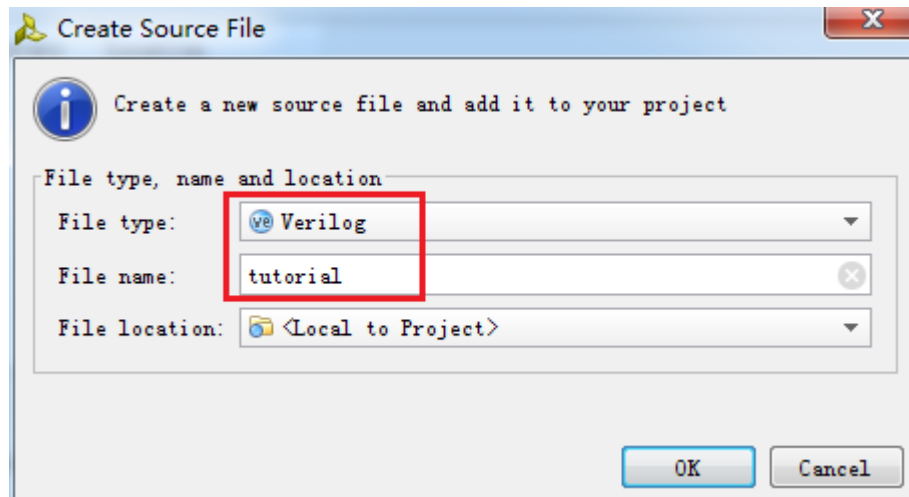
- 2、选择“Add or Create Design Sources”，用来添加或新建Verilog或VHDL源文件。



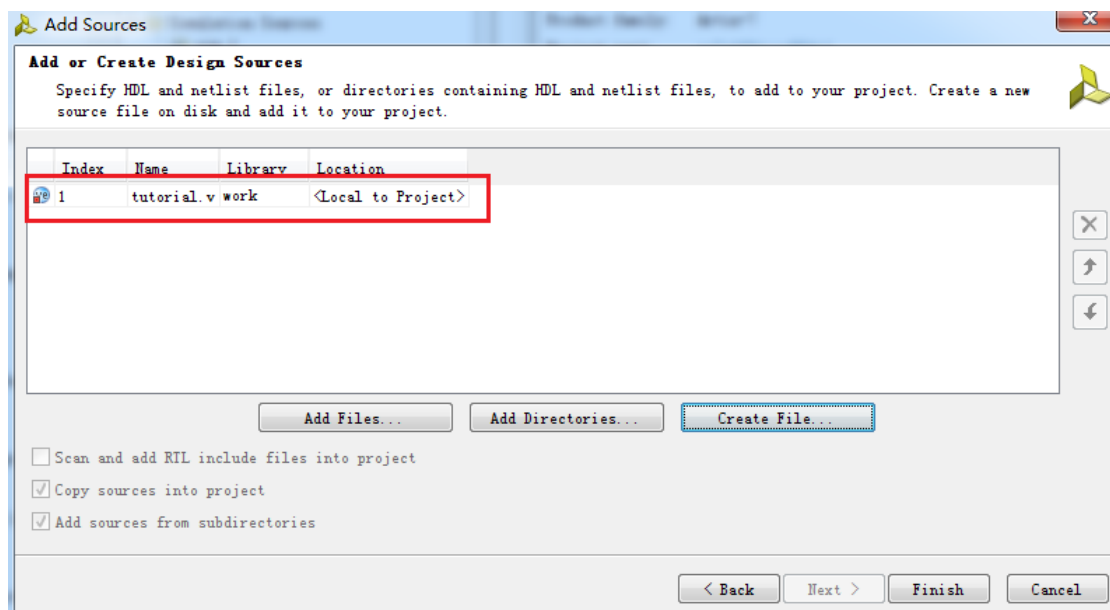
点击“Next”。如果有已经写好的.v/.vhd文件，可以通过“Add Files”一项添加。在这里，我们要新建文件，所以选择“Create File”这一项。



4、在“File Type”中选择Verilog，在“Create Source File”中“File Name”栏输入你为该工程取的顶层实体文件名，这里我们取名为“tutorial”。文件名可以和工程名一样，也可以不一样。**注意：件名称以字母开头，由字母、数字、下划线来组成，不能出现中文和空格，不能以数字开头。**如图。

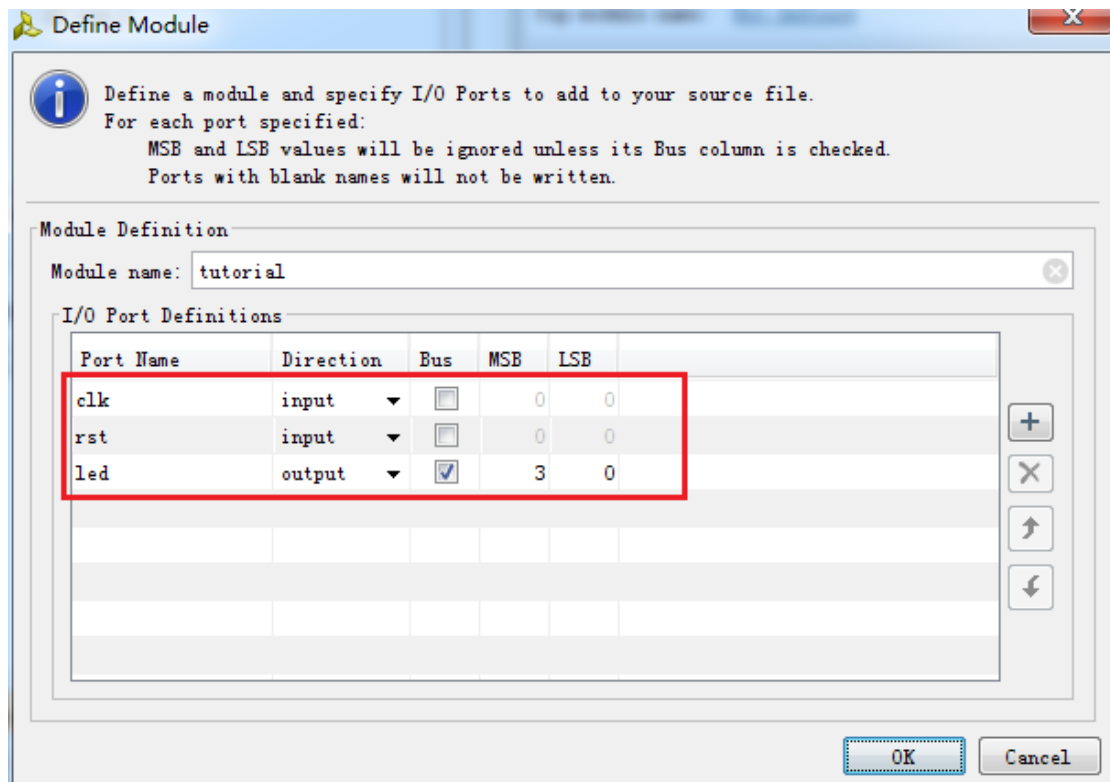


点击“OK”。新添加的Verilog文件“tutorial.v”出现在对话框中。

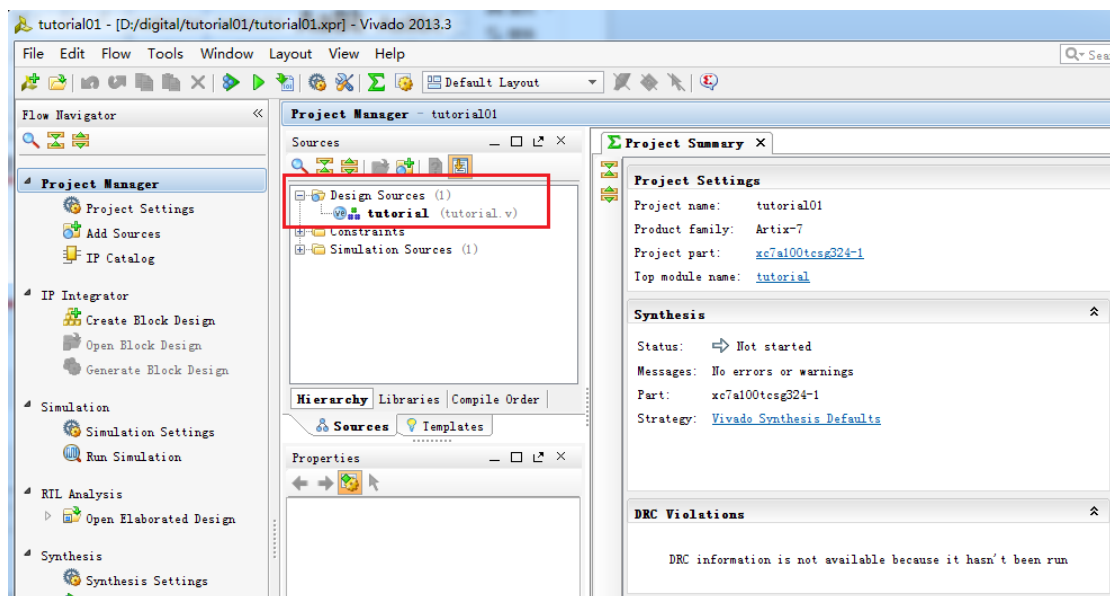


点击“Finish”。

6、在弹出的Define Module中的I/O Port Definition，输入此次设计的模块中所需的端口，并设置输入/输出，如果端口为总线型，勾选Bus选项，并通过MSB和LSB确定总线宽度。完成后点击OK。（这一步也可以不用设置端口线，以后在Verilog文件中直接声明也可以。如果这一步设置错了，也可以在Verilog文件中修改。）



7、新建的设计文件（此处为tutorial.v）即存在于Sources中的Design Sources中。



双击“tutorial.v”打开该文件，输入相应的设计代码。如图。如果刚才设置了端口，则此时的代码中已经对端口进行了声明，如刚才没有设置端口，在这里要重新声明。

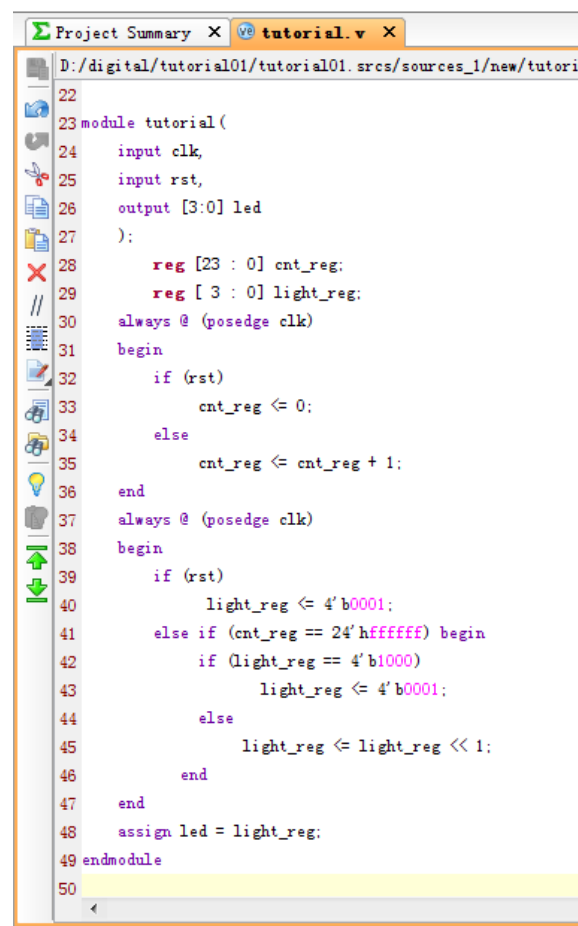
本段代码的功能是完成一个四个灯的循环点亮电路。

这样来说：有一个电路，它有两个输入端clk和rst，rst的功能是对电路进行复位，每次输入“1”值，打开复位开关，四个灯的最右面的等亮，其他灯暗。如

果输入“0”值复位开关“关”则电路正常工作，四个灯循环向左逐个点亮，但每次只有一个灯亮。clk是时钟信号，这里使用的是开发板上自带的一个系统时钟，频率是100MHZ的。当时钟信号从0计满一次 2^{24} 后，亮的灯顺次向左移1位。

电路的输出端是led，是一个四位的变量。复位时输出“0001”，然后每计满个 2^{24} 时钟周期，循环左移一位。

这段代码的两个输入端是：时钟clk和复位rst端，输出端是一个四位的总线结构Led[3:0]（led[3]、led[2]、led[1]、led[0]）。



```
22
23 module tutorial (
24     input clk,
25     input rst,
26     output [3:0] led
27 );
28     reg [23 : 0] cnt_reg;
29     reg [ 3 : 0] light_reg;
30     always @ (posedge clk)
31     begin
32         if (rst)
33             cnt_reg <= 0;
34         else
35             cnt_reg <= cnt_reg + 1;
36     end
37     always @ (posedge clk)
38     begin
39         if (rst)
40             light_reg <= 4'b0001;
41         else if (cnt_reg == 24'hffffff) begin
42             if (light_reg == 4'b1000)
43                 light_reg <= 4'b0001;
44             else
45                 light_reg <= light_reg << 1;
46         end
47     end
48     assign led = light_reg;
49 endmodule
50
```

代码解释：

```
module tutorial(
    input clk,           // 输入信号，时钟信号，缺省宽度，默认为1位
    input rst,           // 输入信号，复位信号，缺省宽度，默认为1位
    output [3:0] led     // 输出信号，宽度为4位，发光二极管
);
    reg [23 : 0] cnt_reg; // 局部变量，1个24位的寄存器，用于计数
    reg [ 3 : 0] light_reg; // 局部变量，1个4位的寄存器，用于暂存数据
    /*因为以上两个变量要在always块中被赋值，因此语法要求定义为reg型变量。
```

如果是在assign语句中被赋值，则应声明为wire型变量。 */

```
always @ (posedge clk) // always块，每当时钟上升沿到来时执行always块
                        // 中begin      end   之间的语句

begin
    if (rst)                // 如果复位信号有效
        cnt_reg <= 0;        // 计数器清0
    else                    // 否则
        cnt_reg <= cnt_reg + 1; // 计数器加1计数
end

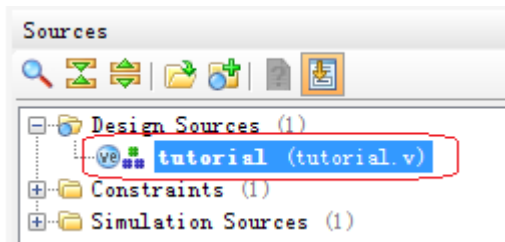
always @ (posedge clk) // always块，每当时钟上升沿到来时执行always
                        // 块中begin      end   之间的语句

begin
    if (rst)                // 如果复位信号有效
        light_reg <= 4'b0001; // 暂存器中的值赋值为 4'b0001
    else if (cnt_reg == 24'hffffff) begin // 如果复位信号无效，切计数器已
                                           // 经计数到24'hffffff
        if (light_reg == 4'b1000) // 如果暂存器中的值为 4'b1000
            light_reg <= 4'b0001; // 则将暂存器赋值为 4'b0001
        else // 如果暂存器中的值不为 4'b1000
            light_reg <= light_reg << 1; // 则将暂存器中的数左移一位
    end
end

assign led = light_reg; // 将暂存器中的值从输出端输出
endmodule
```

如果大家不能理解这段代码的含义，也没有关系，这个例子只是让大家先认识一下Verilog语言，了解一下Vivado设计的流称，轮流点亮灯是为了增加实验的趣味性。

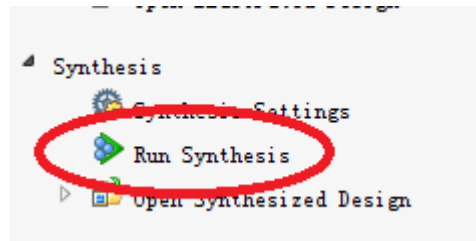
保存，编译器会自动初步检查语法错误，并将此文件加入到工程库中，如果编译器检查没有语法、拼写错误，则在Sources栏可见此文件，



且文件前有一个“”型符号。

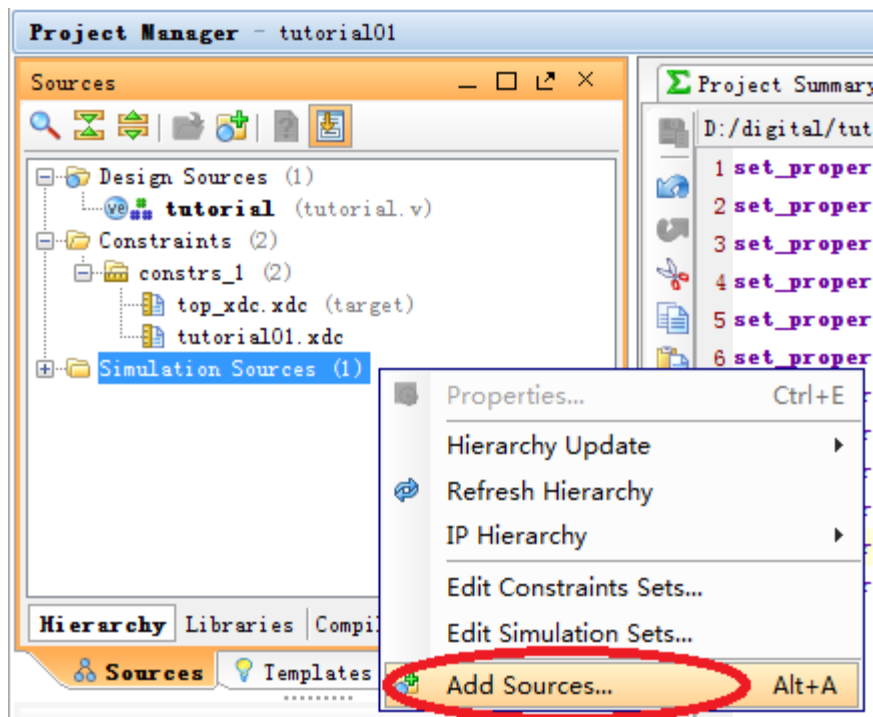
如果没有此符号出现，说明文件没有正确加到工程中，检查添加文件步骤是否正确，或者文件中是否有逻辑错误使得文件无法正常解析。

激励代码保存后，可以点击工程导航栏Synthesis下的“Run Synthesis”，对工程进行编译。编译过程是对设计进行检查，检查设计中的简单逻辑、语法错误等，请仔细检查“error”项，修改每一个error后再编译。

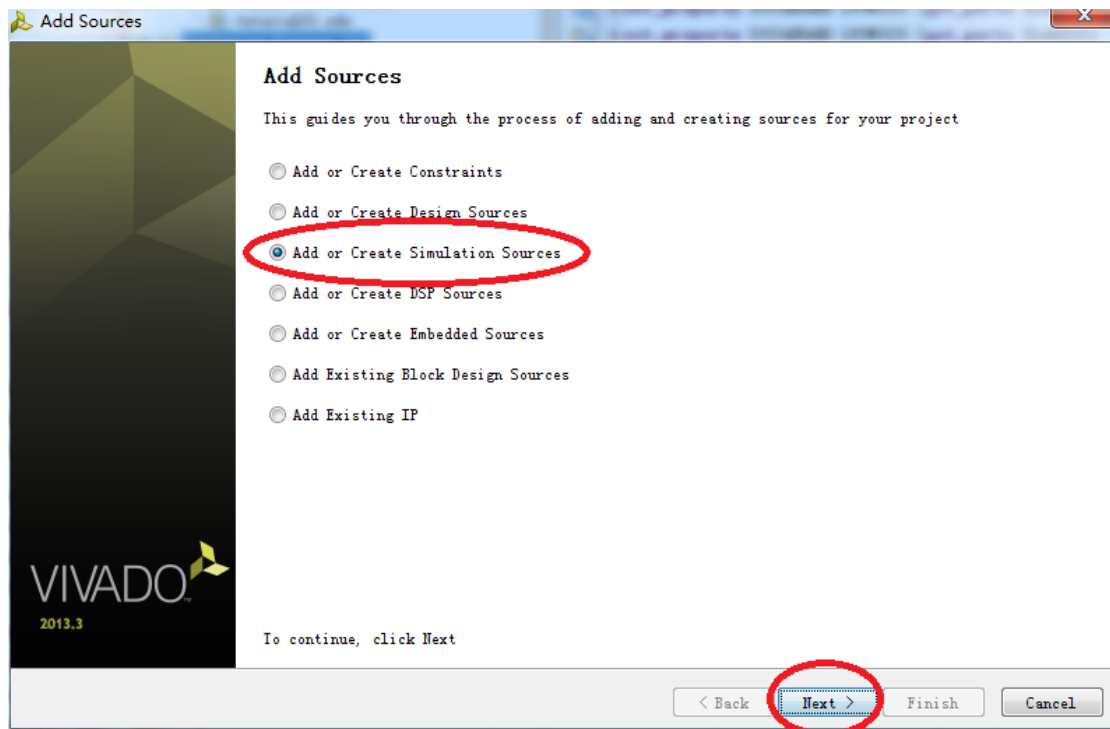


二、利用Vivado进行功能仿真

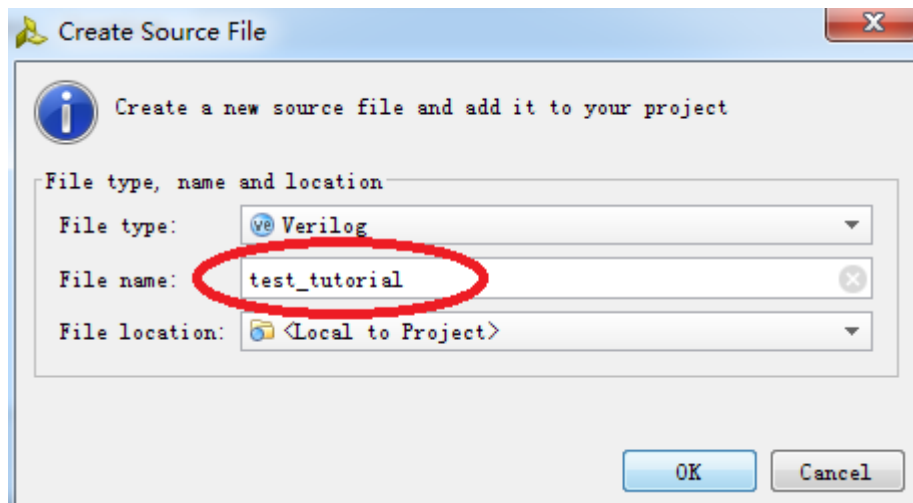
1、创建激励测试文件，在 Source 区域中右击选择 Add source



2、在 Add Source 界面中选择第三项 Add or Create Simulation Sources，点击 Next。

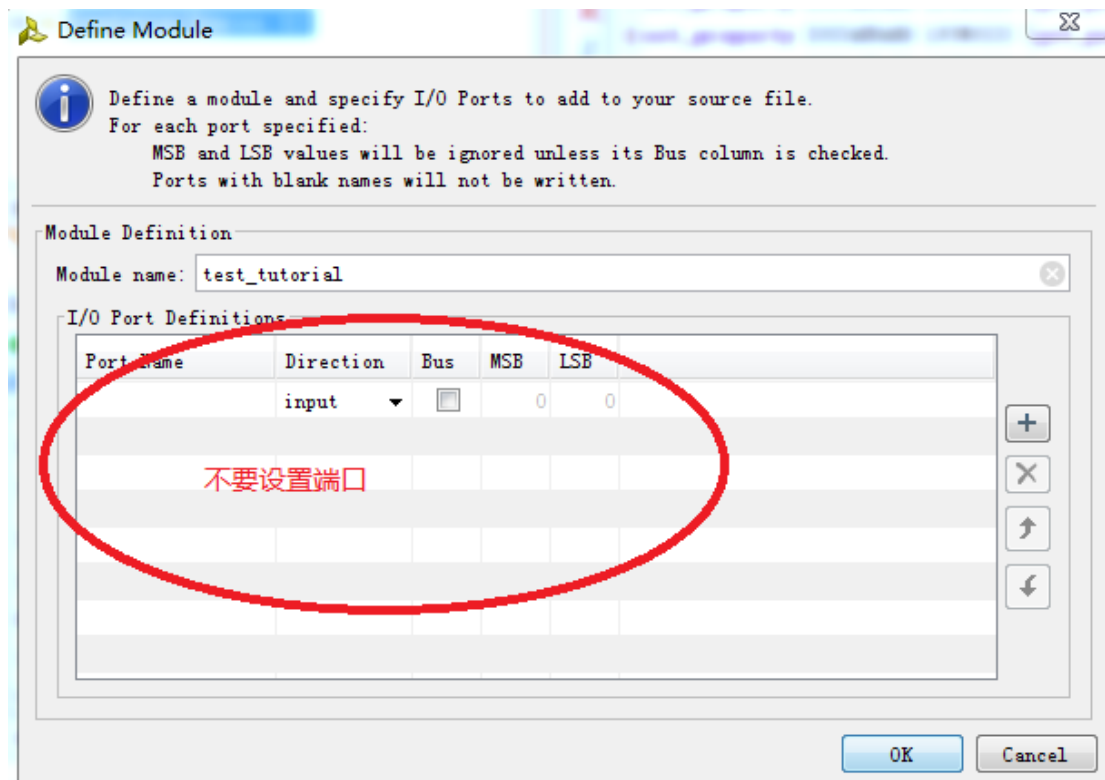


3、选择 Create File，创建一个新的激励测试文件



4、输入激励测试文件名（和设计代码文件区别开），点击 OK。点击 Finish。

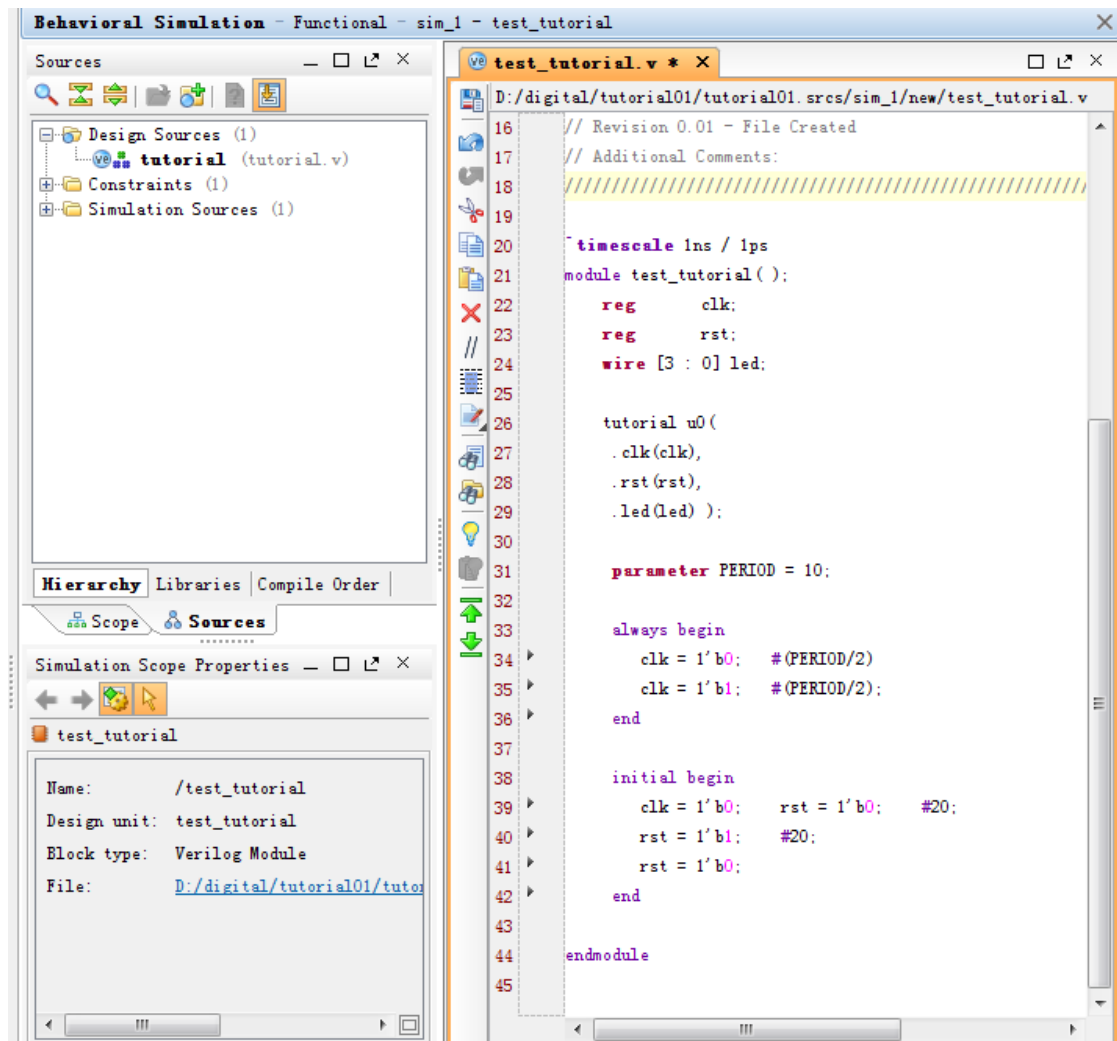
5、在弹出的端口设置对话框中，不要设置端口。



6、点击 OK。

7、新建的测试文件（此处为 test_tutorial.v）即存在于 Sources 中的 Simulation Sources 中。

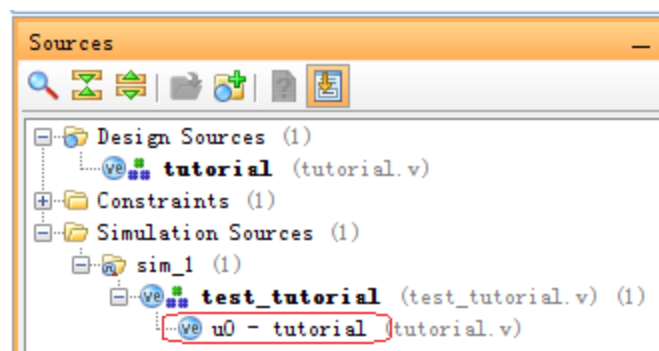
8、双击打开 test_tutorial.v，输入测试激励代码。



激励代码的作用是给输入信号赋不同的值，检查输出的变化是否符合设计要求。

注意：测试代码的端口参数列表要保持空的；在变量声明中，输入变量声明“reg”类型的，输出变量要声明为“wire”类型的。

保存激励代码，如果没有语法错误，这时在Sources栏的Simulation Sources栏下会出现实例“u0”。因为测试代码中将tutorial模块实例化，并取名为“u0”，如果没有出现“u0”，则检查测试代码是否有逻辑错误，修改错误，保存，直至测试代码被正确解析。



激励代码解释：

```
`timescale 1ns / 1ps
```

此语句说明时延时间单位为1 ns并且时间精度为1ps。如“#2”代表延时时间2 ns。

另：在生成test_tutorial.v文件的时候，编译器会在test_tutorial.v文件的第一个自动产生这个语句，如果自动产生了，则不用用户重复输入了，但用户可以修改延时时间和时间精度。

```
module test_tutorial( );
```

```
    reg        clk;
```

```
    reg        rst;
```

```
    wire [3 : 0] led;
```

```
    tutorial u0(
```

```
        .clk(clk),
```

```
        .rst(rst),
```

```
        .led(led) );
```

/*这条语句是将tutorial.v文件中的tutorial模块实例化，并取名为“u0”。因为我们是要对tutorial模块进行仿真，因此要实例化tutorial模块。*/

```
    parameter PERIOD = 10;
```

//定义一个常量

```
    always begin          /* “always”是指循环执行下面这段激励，一般用于写时钟激励，也可以将下面两句写为： clk1 = ~clk1;  #(PERIOD/2);    */
```

```
        clk = 1'b0;      #(PERIOD/2);  //让clk = 0，维持5ns
```

```
        clk = 1'b1;      #(PERIOD/2);  //让clk = 1，维持5ns
```

```
    end
```

```
    initial begin          // 初始化 ， 将所有变量初始化初值
```

```
        clk = 1'b0;      rst = 1'b0;    #20;
```

```
        /* 将clk初始化为0，rst初始化为0，维持20ns */
```

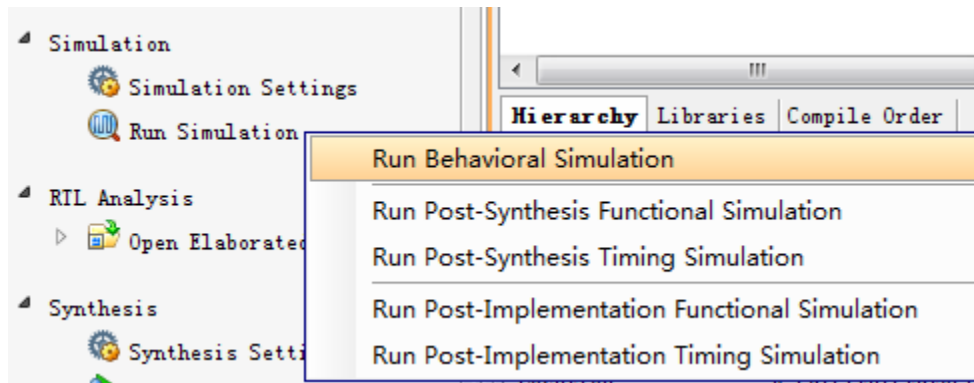
```
        rst = 1'b1;      #20;  /* 将rst设置为1，维持20ns */
```

```
        rst = 1'b0;      // 然后将rst设置为0
```

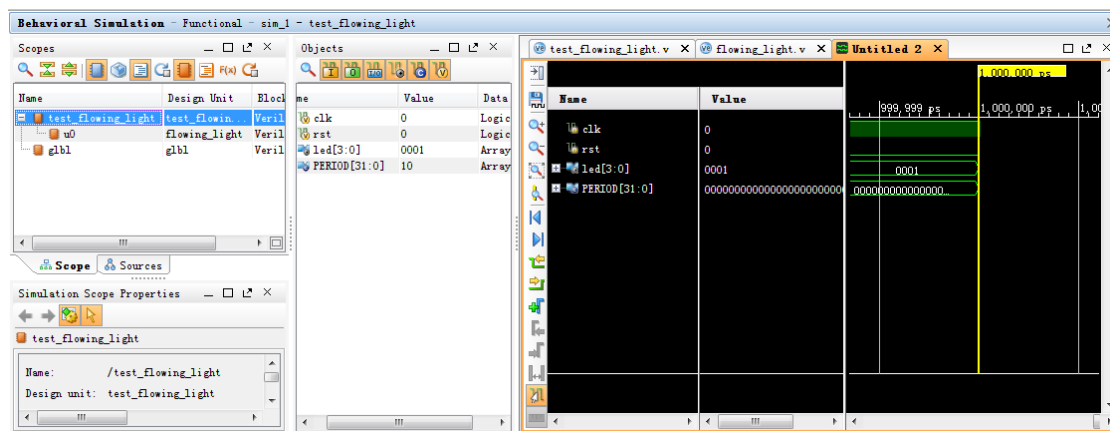
```
    end
```

```
endmodule
```


保存激励文件后，点击项目导航栏Simulation下的“Run Simulation”，选择Run Behavioral Simulation



9、下图所示为仿真界面。



可以点击等图标，对仿真图进行放大和缩小。

还可以通过左侧 Scope 一栏中的目录结构定位到设计者想要查看的 module 内部寄存器，点击 Scopes 栏中的“u0”，在 Objects 对应的信号名称上右击选择 Add To Wave Window，将信号加入波形图中。



也就是在 FPGA 开发平台上的这块具体的 FPGA 芯片内实现我们设计的电路，如果我们想验证我们的电路运行是否正确，在其输入端输入变量，检测输出结果是否和我们的设计值一致，那该如何做呢？

我们设计的电路是在被下载到FPGA上实现的，FPGA自身没有输入输出设备，必须使用外部的输入输出设备。在Nexys4开发平台上，所有的输入输出设备都被固定地和FPGA的某个引脚连接在一起了。比如Nexys4开发板上的发光二极管LD0，LD1，LD2和LD3分别和FPGA的T8，V9，R8和T6连接在一起的（这个在发光二极管的旁边有标识）。我们的电路在FPGA中实现的，我们的输出端口取名为LED[0]，LED[1]，LED[2]和LED[3]。如果我们将FPGA内部的LED[0]和FPGA的T8引脚连接起来，而T8在外部又是和LD0连接起来的，那么在硬件上我们就将LED[0]和LD0连接起来了，如果从我们设计的电路的LED[0]端输出“1”值，开发板上的LD0就会亮。这样就可以使用外部的输入输出设备来验证我们的电路了。

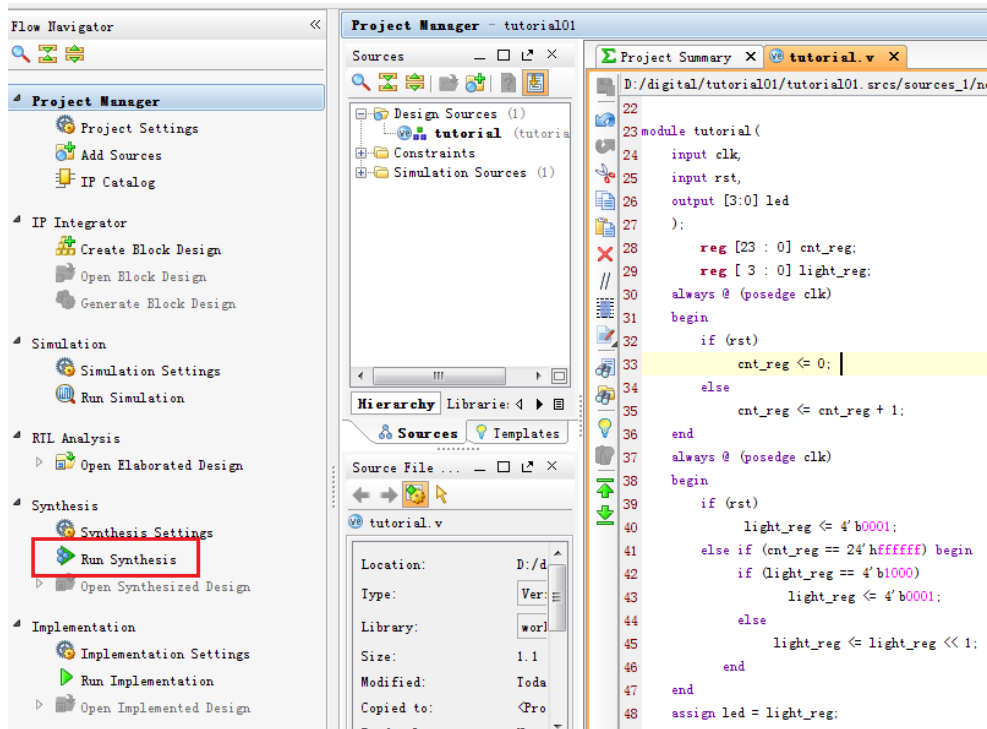
另外，在FPGA开发平台上，FPGA芯片的“E3”引脚和平台上的100MHZ的一个系统时钟相连。而在我们的电路中，有一个输入信号“clk”就是需要一个100MHZ的时钟，那我们就将我们声明的变量“clk”和FPGA的引脚“E3”连接，而“E3”在开发板的硬件上又是和100MHZ的系统时钟相连的，这样，我们的输入变量就有一个100MHZ的时钟输入了。

如何配置我们设计的电路中其他的输入/输出引脚，利用开发板上的输入输出硬件来验证我们设计的电路呢？这就需要对我们的电路进行引脚约束，将我们电路中的输入输出端和FPGA的引脚进行配置连接，从而使其连接到开发板的输入输出器件上。

有两种方法可以添加引脚约束文件，一是可利用Vivado中IO planning功能，二是可以直接新建XDC的约束文件，手动输入约束命令。**选择其中一种方法即可，不需要建立两个文件。**

先来看第一种方法，利用IO planning。

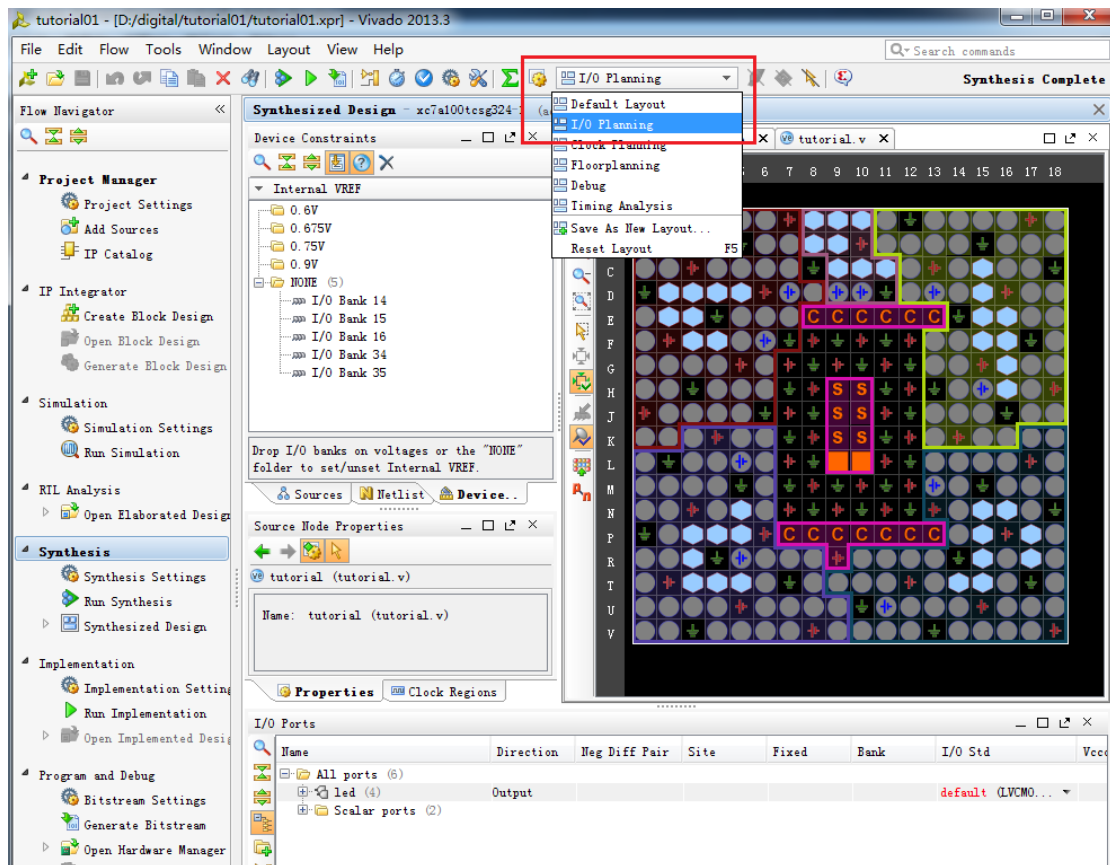
如果前面工程没有综合Run Synthesis，则点击Flow Navigator中Synthesis中的Run Synthesis，先对工程进行综合。如果前面的设计有问题，会在综合过程中报错，修改错误，再综合。综合成功后，才能进行引脚设置。如果前面已经综合完成，则不需要重复综合。



综合完成之后，选择Open Synthesized Design，打开综合结果。



此时应看到如下界面，如果没出现如下界面，在图示位置的layout中选择IO planning一项。



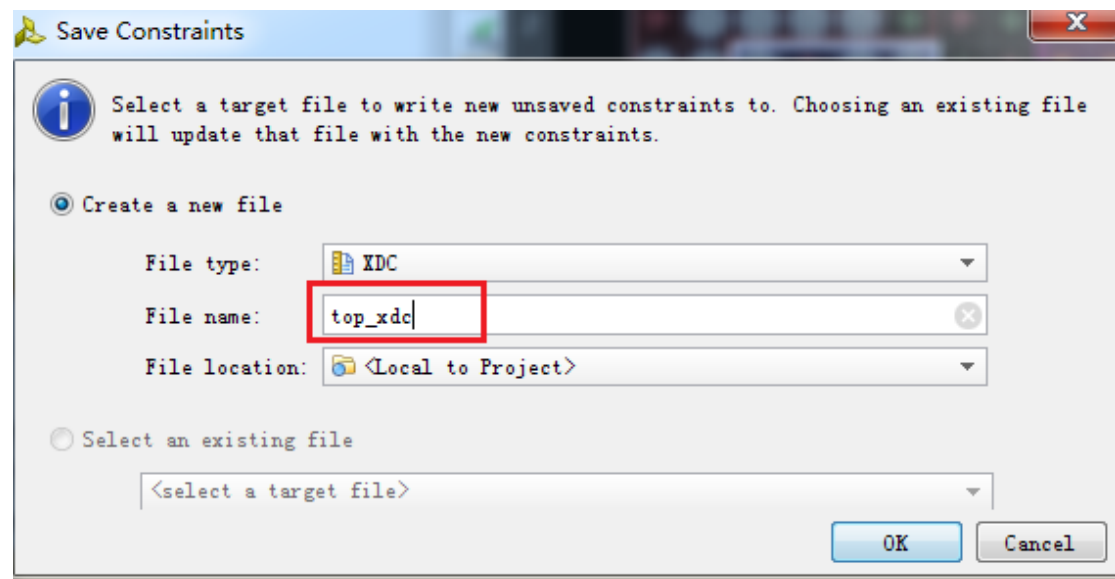
在右下方的选项卡中切换到I/Oports一栏，并在对应的信号后，输入对应的FPGA管脚标号（或将信号拖拽到右上方Package图中对应的管脚上），并指定I/O std。（具体的FPGA约束管脚和IO电平标准，可参考对应板卡的引脚用户手册“Nexys4_Master.xdc”文件或原理图“Nexys4_RM_VB1_Final_3.pdf”文件）。或者有些引脚直接在开发板上已经清楚标注了）。

Name	Direction	Neg Diff Pair	Site	Fixed	Bank	I/O Std	Vcco	Vref
All ports (6)								
led (4)								
led[3]	Output		T8	☒		34 LVCMOS33*	3.300	
led[2]	Output		V9	☒		34 LVCMOS33*	3.300	
led[1]	Output		R8	☒		34 LVCMOS33*	3.300	
led[0]	Output		T6	☒		34 LVCMOS33*	3.300	
Scalar ports (2)								
clk	Input		E3	☒		35 LVCMOS33*	3.300	
rst	Input		U9	☒		34 LVCMOS33*	3.300	

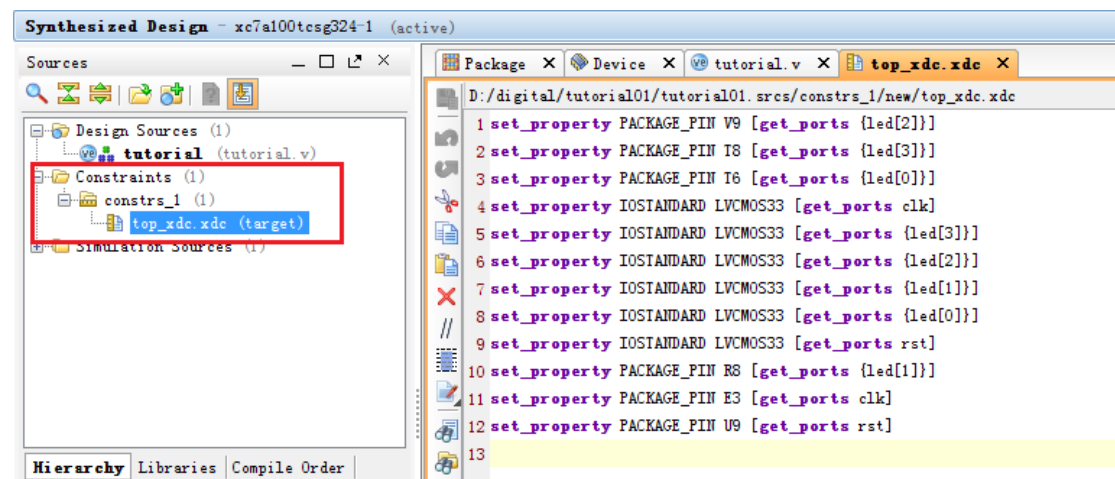
完成之后，点击左上方工具栏中的保存按钮，工程提示新建XDC文件或选择工程中已有的XDC文件。在这里，我们要Create a new file，输入Filename，点击OK完成约束过程。

重复解释：将输出端led[3]和FPGA的引脚“T8”连接在一起。实际上，在开发板上，T8引脚是在硬件上和开发板上提供的输出显示发光二极管LD3连接在一起的。因此此引脚配置的意思是：将led[3]通过FPGA的T8引脚和开发板上的发光二极管LD3连接在一起，如果led[3]输出高电平，则LD3亮，如果led[3]输出低电

平，则LD3暗。另外，引脚电平设置是此引脚的电平标准设为3.3V。

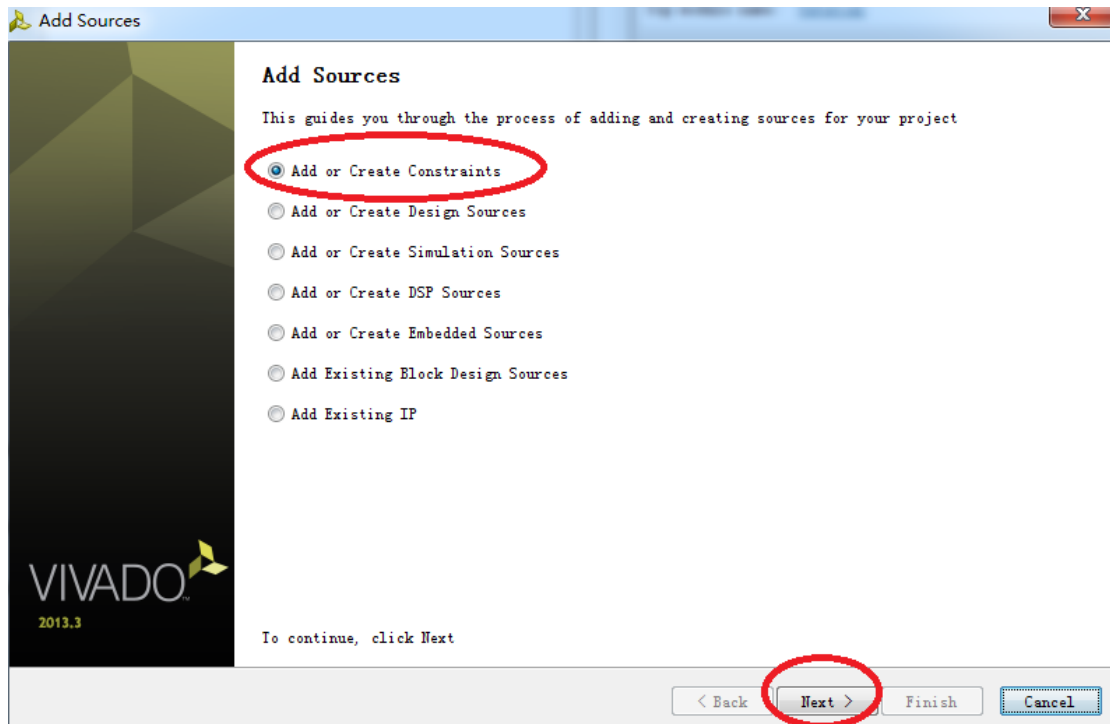


此时，在Sources下Constraints中会找到新建的XDC文件。

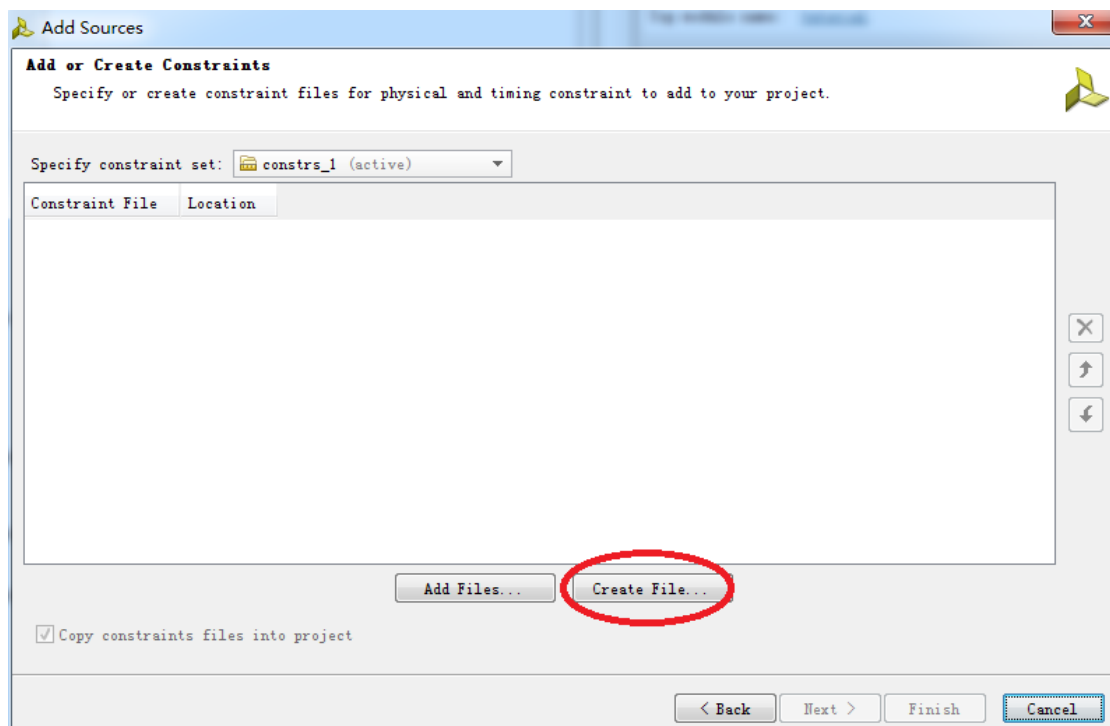


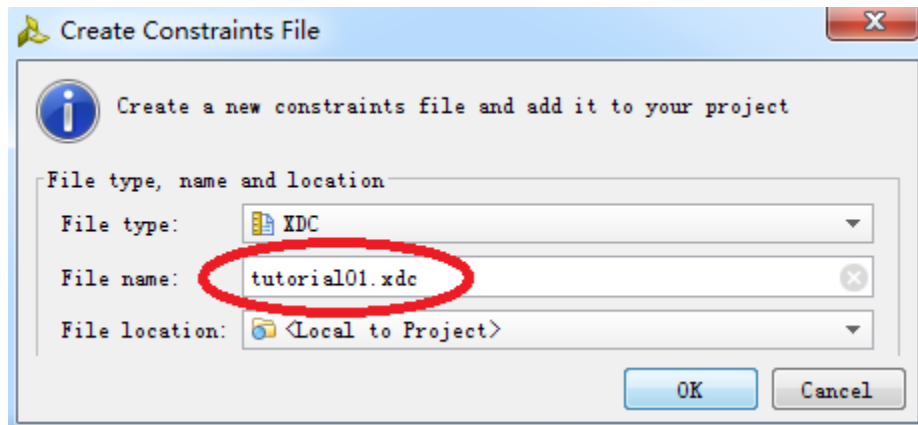
如何用第二种方法添加约束文件

点击Add Sources，选择第一项Add or Create Constraints一项，点击Next。

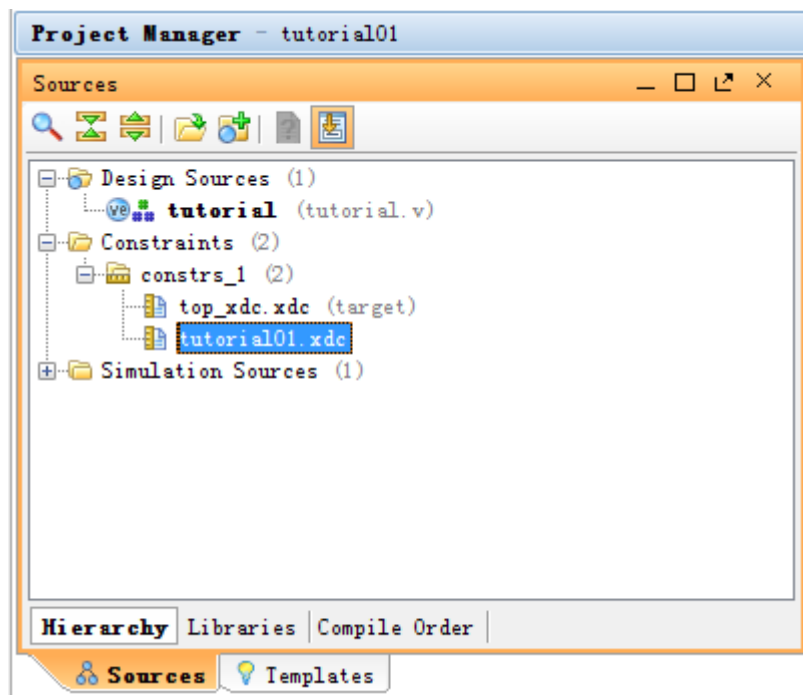


点击Create File，新建一个XDC文件，输入XDC文件名，点击OK。点击Finish。





双击打开新建好的.xdc文件，并按照如下规则，输入相应的FPGA管脚约束信息和电平标准。



```
set_property PACKAGE_PIN T8 [get_ports {led[3]}]
set_property PACKAGE_PIN V9 [get_ports {led[2]}]
set_property PACKAGE_PIN R8 [get_ports {led[1]}]
set_property PACKAGE_PIN T6 [get_ports {led[0]}]
set_property PACKAGE_PIN E3 [get_ports clk]
set_property PACKAGE_PIN U9 [get_ports rst]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
```



```
set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports rst]
```

请坚持引脚设置，检查是否有遗漏，如果有管脚设置遗漏，则产生二进制文件时会报错。

请查看一下“Nexys4_Master.xdc”文件，思考其他项目如何设置该文件。

解释：

```
set_property PACKAGE_PIN V9 [get_ports {led[2]}]
```

这个引脚约束的意思是：将我们程序中的输入端“led[2]”接到FPGA

“xc7a100tcsg324-1”的“V9”引脚上。在NEXYS4开发板上，这个引脚是和开发板上的发光二极管LD2连接在一起的，也即，输出信号led[2]将通过

“xc7a100tcsg324-1”的“V9”引脚和发光二极管LD2连接在一起，如果led[2]输出“1”则发光二极管LD2亮，如果led[2]输出“0”则发光二极管LD2暗。

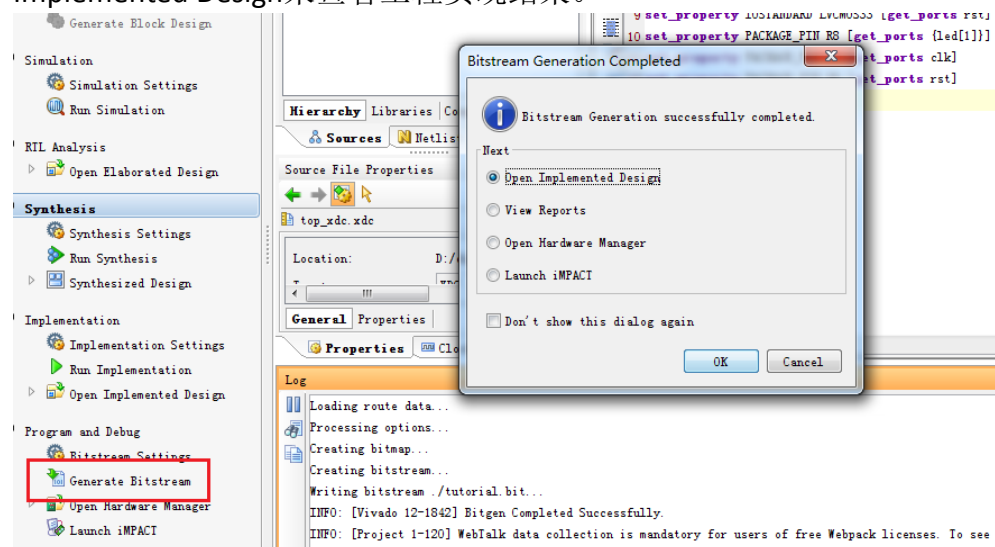
```
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
```

这个引脚约束的意思是：设置引脚电平为3.3v。

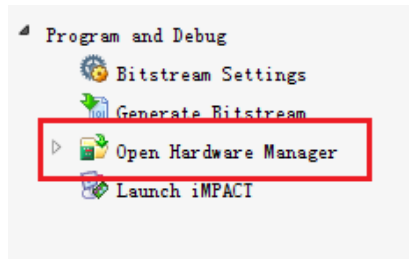
将我们的设计配置到开发板上，形成硬件电路：

将NEXYS4开发板通过USB口连接到电脑上，等待自动安装驱动程序，约1-2分钟。

1、在Flow Navigator中点击Program and Debug下的Generate Bitstream选项，工程会自动完成综合、实现、Bit文件生成过程，完成之后，可点击Open Implemented Design来查看工程实现结果。

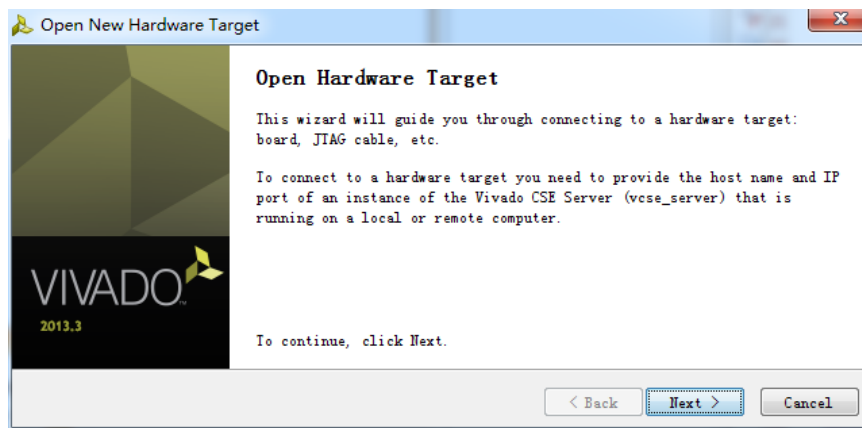


2、点击Flow Navigator中Open Hardware Manager一项，进入硬件编程管理界面。

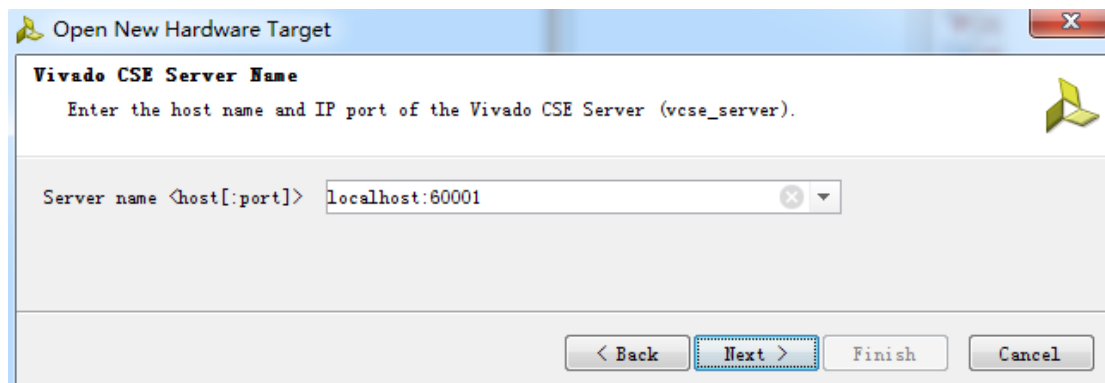


3、在提示的信息中，选择Open a new hardware target（或在Flow Navigator中展开Hardware Manager，点击Open New Target），

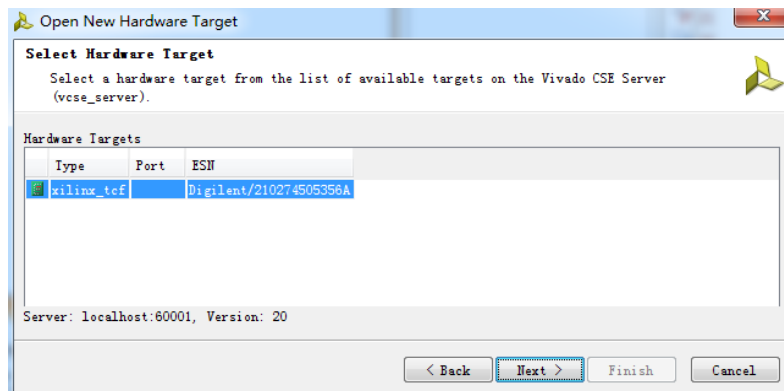
4、在弹出的Open hardware target向导中，先点击Next，进入Server选择向导。



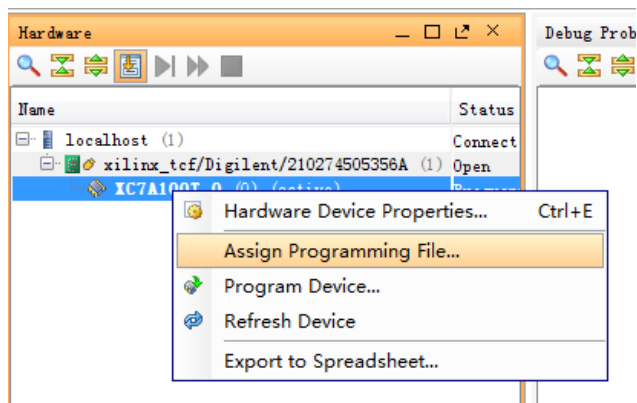
5、保持默认的Server name为local host: 60001，如无默认，在下拉框中选择即可。连接好板卡的PROG端口，并上电。点击Next。



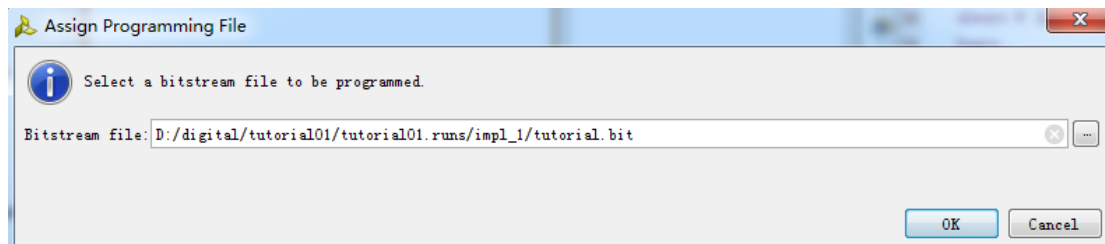
6、依次点击Next、Next、Finish完成新建Hardware Target。



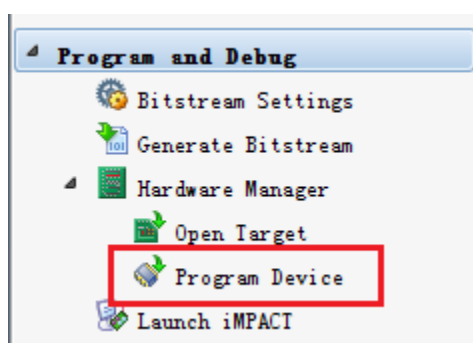
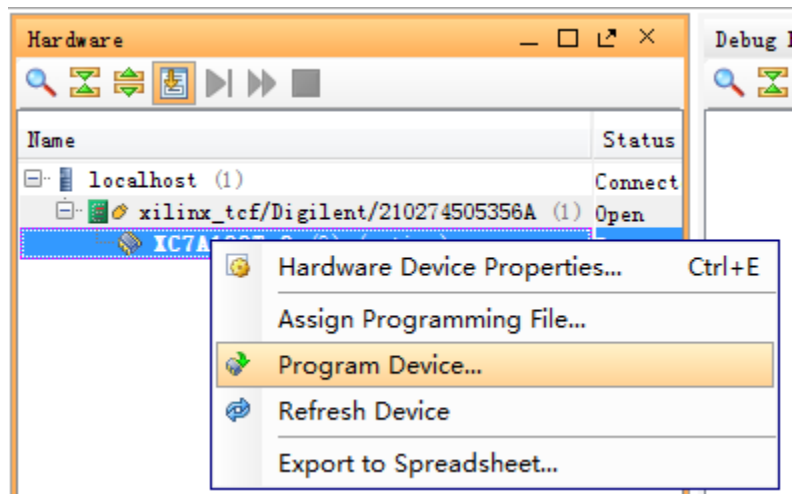
7、此时，Hardware一栏中出现硬件平台上可编程的器件XC7A100T_0。在对应的FPGA器件上右击，选择Assign Programming File，指定所需的bit文件。（系统默认已存在该工程的bit文件，如不需更改，可跳过该步骤）



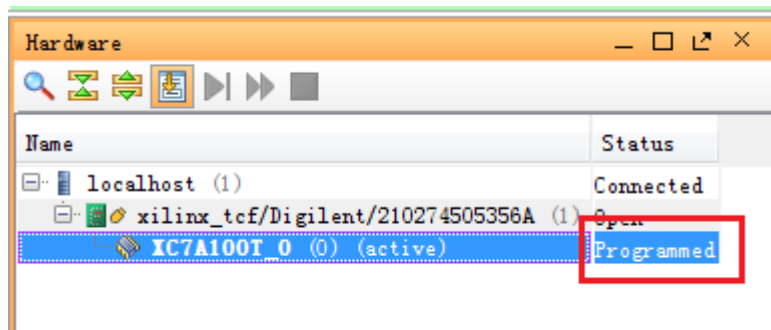
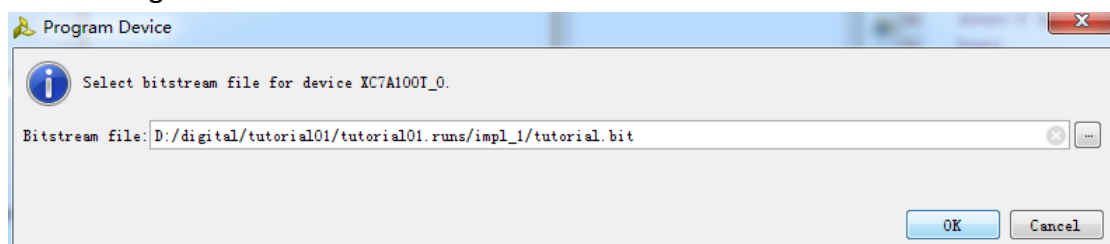
如果没有出现“Assign Programming File...”，直接点击“Program Device...”。



8、在FPGA器件上右击选择Program Device或在Flow Navigator下的Hardware Manager中点击Program Device一项。



9、点击OK，将bit文件下载到板卡上的FPGA中，此时Hardware对应的FPGA状态即变为Programmed。



10、 硬件验证设计完成。

观察NEXYS4开发板的LD0—LD3，是否有一个灯循环点亮。

思考：如果将灯换成LD10—LD13，如何实现？